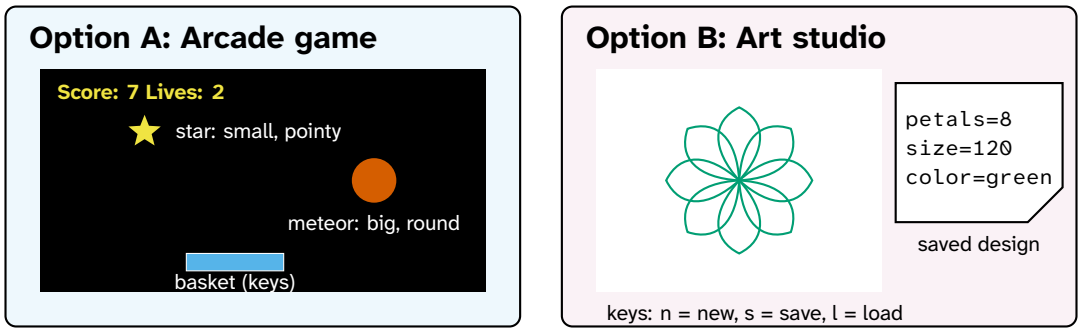
Final Project Brief

Arcade Game or Art Studio: choose, plan, build, test and show (weeks 9–12)

Your mission: you have learned functions, lists, dictionaries, loops, files, tests and games with keys and timers. Now you put them together in a program that is completely **yours**. In week 12 your family will come to our **Family Arcade** to play your game or make art with your studio, and you will explain how it works. You can work alone or with a partner.

Step 1: Choose your option



	Option A: Arcade game	Option B: Art studio
What it is	A turtle game with **keys**, a **timer** and a **score**.	A program that **makes designs** (often random) and can **save** and **load** them.
You will love it if...	you like playing and inventing games.	you like patterns, colors and surprises.
Starter file	`starter/game_template.py`	`starter/art_studio_template.py`
The file saves...	high scores	designs (one setting per line)

Step 2: Know the must-haves

Whatever you choose, your finished program must have all six. Check them off as you go.

- ☐ **Functions** with parameters that **return** a value (for example `move(x, step)` or `is_touching(a, b)`).
- ☐ A **list** (enemies, coins, colors, a gallery of saved designs...).
- ☐ A **dictionary** (settings, a player, a design...).
- ☐ A **while loop** or a **game loop with** `ontimer`.
- ☐ A **file**: high scores or saved designs, next to your program, created if it is missing.
- ☐ At least **3 pytest tests** of your own functions, in `my_tests.py`.

> **Tip: The secret of easy tests**
>
> Put the *rules* of your program (scores, moving, touching, saving) in small functions that do **not** use turtle. They take values in and return an answer, so they are easy to test. Keep the drawing in other functions.

Step 3: Get an idea

Pick one from the menu, change it, or invent your own.

Idea	What happens	A list of...	The dictionary / file
Arcade games			
Space Snack	Fly a ship left and right, catch snacks, dodge asteroids.	falling snacks and asteroids	settings; high scores
Dragon Feeder	Move a dragon to eat apples before the timer ends. Golden apples are worth more.	apples on the screen	points for each kind; high scores
Rainy Day	Move an umbrella to keep a cat dry. Rain falls faster over time.	raindrops	speed levels; best time

Idea	What happens	A list of…	The dictionary / file
Bug Squasher	Bugs appear in random places; move a net onto them with the arrow keys.	bugs	bug types and points; high scores
Treasure Diver	A diver collects pearls while sharks swim side to side.	sharks and pearls	settings; top 5 scores
Art studios			
Kaleidoscope Lab	Random mirrored patterns; press keys to change the number of mirrors.	colors, shapes	one design; saved designs file
Pixel Pet Maker	Draw a pet on a grid of squares from rows of letters.	rows of the grid	letter → color; saved pets
Spiro Studio	Spirograph-style loops; keys change size and turn.	a gallery of designs	one design; saved gallery
Quilt Maker	A quilt of squares, each with a random pattern.	patterns	pattern settings; saved quilts
Firework Show	Click or press keys to launch fireworks; save your show and replay it.	fireworks in the show	one firework; saved show

Milestone	When	Done when…	✔

Step 4: Milestones

Big projects are built in small steps. Show your teacher each milestone.

Milestone	When	Done when…	✔
M1 Pitch and plan	end of week 9	Your pitch page, your pseudocode or flowchart, and (in a pair) your task split are filled in.	☐
M2 Core loop works	week 10	The main thing happens on screen: the player moves and things fall, or the studio draws a design when you press a key.	☐
M3 Data and tests	week 11, first half	Your list, dictionary and file work. `my_tests.py` has 3 tests that pass.	☐
M4 Polish	week 11, second half	Someone playtested it, you fixed one thing they found, and you did the accessibility check.	☐
Showcase	week 12	You showed it at the Family Arcade and explained one part.	☐

Planning page 1: My pitch (M1)

Name (nickname is fine): _____ Date: _____

Project name: _____ Option: A ☐ B ☐

In one sentence, my project is...

How do you play it or use it? (keys, goal, how it ends)

My **list** will hold...

My **dictionary** will hold...

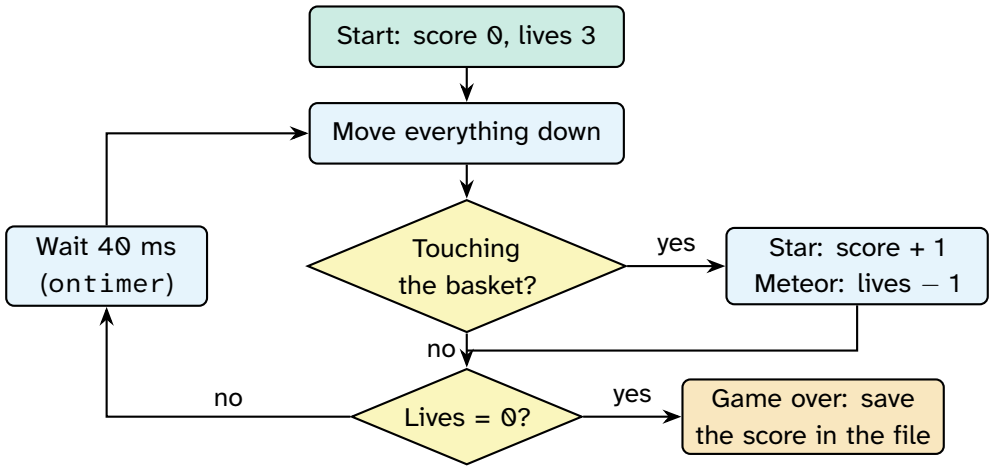
My **file** will save...

My **loop** (`while` or `ontimer`) will...

One thing that will make it special:

Planning page 2: Pseudocode or flowchart (M1)

Plan the steps in plain words or boxes and arrows before you write Python. Here is the plan for a catching game:



My plan (pseudocode or flowchart):

Planning page 3: Task split for pairs (M1)

Split the work fairly: each person gets interesting jobs, and each person writes at least one function **and** one test. Swap driver and navigator every 8 minutes as usual.

Task	Who	Done
Rules functions (move, touching, score, ...)		☐
Drawing: shapes, colors, text on screen		☐
Keys and timer (`onkey`, `ontimer`) or key menu		☐
The list and the dictionary		☐
Saving and loading the file		☐
Tests in `my_tests.py`		☐
Playtest notes and accessibility check		☐
		☐

Planning page 4: My tests (M3)

Test a **normal** case and at least one **edge** case (a wall, zero, an empty list, a missing file).

Function	I give it…	It should return…	Normal / edge
`clamp` (example)	`500, -270, 270`	`270`	edge

Planning page 5: Playtest and accessibility (M4)

- ☐ A playtester tried it while I watched quietly and took notes.
- ☐ I fixed or improved one thing from the playtest: _____
- ☐ Colors come from the safe palette, and things that matter also differ in **shape** or **size**.
- ☐ Text is big enough to read from a chair.
- ☐ The keys are easy to reach, and the screen says which keys to use.
- ☐ It never asks for a real name, age, address, school or password.
- ☐ If I used someone else's idea, art or code, I wrote who made it in a comment.

How your project is assessed

Your teacher uses the same four questions. Check where you think you are now, and again at the end.

	Just starting	Getting there	Got it	Expert
Creativity	I changed a few things in the template.	My project has my own theme and one new idea.	My project has its own idea, rules and look.	My idea has a surprising twist that makes people want another go.
Working code	It runs, but crashes or the main part does not work yet.	The main part works; some must-haves are missing.	It works as my pitch said, with all six must-haves.	It also handles tricky moments (walls, missing file, playing again).
Code quality and tests	My code is one long block; no tests yet.	I use some functions; 1–2 tests pass.	Small functions with clear names and comments; 3 or more tests pass.	Rules are separate from drawing; my tests include edge cases.
Explaining it	I can show what it does.	With help, I can explain what one function does.	I can explain how one part works and one decision I made.	I can explain my design, what I would change, and a bug my tests caught.

Support and extension

Differentiation

↓ Support path (needs more time): Start from the template and change it step by step. Aim first for M2 (the core loop) and two tests; your teacher can be your navigator. A small project that works and that you can explain is a great project.

↑ Extension path (finished early): Put your rules in their own file (like `starcatcher_logic.py` in the reference game) and import it. Add levels, a start menu, a top-5 high-score table, or let players choose their own keys. Write a test for your file functions.

When you are stuck

1. Read the error message: start with the **last line**.
2. Print the values: `print("x is", x)`.
3. Explain your code line by line to your partner or a rubber duck.
4. Make the problem smaller: test one function on its own.
5. Check the cheat sheet and the `starter/README.md`.
6. Ask your teacher: say what you expected and what happened instead.

Digital citizenship & safety: Showcase safety

Use a nickname on screen and on your project card. Your project stays on our computers and is shown to family only: nothing is uploaded. Credit anyone whose ideas, art or code you used.